

Sección 2: Implementar el uso de herramientas y la interacción del entorno (20-25%)

⚠ **Esta es la sección de mayor peso en el examen.** Cubre desde la selección y configuración de herramientas hasta la integración segura de agentes en entornos de desarrollo, pasando por MCP (Model Context Protocol) y patrones robustos de control de errores.

2.1 Selección y configuración de las herramientas del agente

Conceptos clave

¿Qué son las "herramientas" de un agente?

En el contexto de agentes de IA, una **herramienta (tool)** es cualquier capacidad externa que el agente puede invocar para realizar acciones más allá de la generación de texto. Las herramientas son el mecanismo mediante el cual un agente **interactúa con el mundo real**: leer archivos, ejecutar comandos, consultar APIs, crear PRs, etc.

El modelo mental es: **un agente sin herramientas es solo un chatbot; un agente con herramientas es un actor autónomo.**

Las herramientas en el ecosistema de GitHub Copilot se dividen en categorías:

Categoría	Descripción	Ejemplos
Herramientas integradas (built-in)	Vienen preconfiguradas con el agente	Lectura/escritura de archivos, ejecución de terminal, búsqueda de código
Herramientas de extensión (MCP)	Se agregan mediante servidores MCP	APIs externas, bases de datos, servicios en la nube
Herramientas de flujo de trabajo	Integración con CI/CD y procesos	GitHub Actions, checks, deployments
Herramientas de contexto	Proporcionan información al agente	Archivos de instrucciones, documentación del repo

Identificación de las herramientas necesarias

El proceso de selección de herramientas sigue un enfoque basado en **capacidades requeridas vs. disponibles**:

Paso 1: Analizar la tarea del agente

- ¿Qué necesita leer? (código fuente, documentación, issues)
- ¿Qué necesita escribir? (código, configuración, comentarios)
- ¿Qué necesita ejecutar? (tests, builds, linters)

- ¿Con qué servicios externos necesita comunicarse? (APIs, bases de datos)

Paso 2: Mapear a herramientas disponibles

Para **Copilot coding agent** (agente en la nube), las herramientas integradas incluyen:

- **git** — operaciones de control de versiones
- Terminal/shell — ejecución de comandos
- Lectura y escritura de archivos del repositorio
- Creación de ramas y Pull Requests
- Interacción con GitHub API (issues, PRs, comments)

Para **Copilot en VS Code (modo agente)**, las herramientas incluyen:

- Exploración del workspace (archivos, carpetas)
- Terminal integrada
- Extensiones de VS Code como herramientas
- Servidores MCP configurados por el usuario

Paso 3: Identificar brechas y configurar herramientas adicionales

Si la tarea requiere capacidades que no están en las herramientas integradas, se deben agregar herramientas externas. Esto se hace principalmente a través de **servidores MCP** (ver sección 2.2).

Ejemplo de análisis de brechas:

Tarea: "Crear un endpoint API y documentar en Confluence"

Herramientas necesarias:

- ☒ Lectura/escritura de archivos → Integrada
- ☒ Ejecución de tests → Integrada (terminal)
- ☒ Creación de PR → Integrada
- ☒ Publicar en Confluence → Requiere servidor MCP de Confluence

Configuración de herramientas del agente

Archivo **.github/copilot-instructions.md**

Este archivo es el mecanismo principal para proporcionar instrucciones persistentes al agente a nivel de repositorio. Funciona como una "guía de onboarding" que el agente lee antes de cada tarea.

```
<!-- .github/copilot-instructions.md -->

# Instrucciones para Copilot

## Estilo de código
- Usar TypeScript estricto (strict: true)
- Seguir el patrón de arquitectura hexagonal
```

- Los tests deben usar Jest con cobertura mínima del 80%

Convenciones del proyecto

- Los nombres de archivo usan kebab-case
- Las funciones exportadas usan camelCase
- Los tipos e interfaces usan PascalCase

Herramientas y dependencias

- Usar npm (no yarn ni pnpm)
- Base de datos: PostgreSQL con Prisma ORM
- No instalar dependencias nuevas sin justificación

Restricciones

- NO modificar archivos en /infrastructure/
- NO cambiar la configuración de CI/CD
- Siempre ejecutar `npm test` antes de hacer commit

Archivos `agent.md` (instrucciones específicas de agente)

En VS Code, se pueden crear archivos `.github/agents/*.md` para definir agentes especializados con herramientas y comportamientos específicos:

```
<!-- .github/agents/api-developer.md -->
---
name: api-developer
description: Agente especializado en desarrollo de APIs REST
tools:
  - name: github
  - name: filesystem
  - name: terminal
---

# API Developer Agent

Eres un agente especializado en desarrollo de APIs REST.

## Reglas
- Siempre crear tests de integración para cada endpoint
- Usar el patrón controller-service-repository
- Documentar cada endpoint con OpenAPI/Swagger
```

Archivo `copilot-setup-steps.yml`

Este archivo configura el **entorno de ejecución** del Copilot coding agent (agente en la nube). Define los pasos de preparación que se ejecutan antes de que el agente comience a trabajar:

```
# .github/copilot-setup-steps.yml
steps:
  - name: Instalar dependencias
    run: npm ci

  - name: Configurar base de datos de test
    run: |
      cp .env.example .env.test
      npm run db:migrate:test

  - name: Verificar que el entorno funciona
    run: npm test -- --bail

  - name: Instalar herramientas adicionales
    run: |
      npm install -g @openapi-generator/cli
      pip install pre-commit
```

Puntos importantes sobre `copilot-setup-steps.yml`:

- Se ejecuta en la **VM sandbox** donde opera el Copilot coding agent.
- Los pasos se ejecutan **antes** de que el agente comience su tarea.
- Se usa para instalar dependencias, configurar el entorno, preparar datos de test.
- Es análogo a la sección `steps` de un workflow de GitHub Actions.
- Si un paso falla, el agente no podrá realizar la tarea correctamente.

Configuración de permisos de herramientas de agente

Los permisos controlan **qué puede hacer** el agente y son una capa crítica de seguridad:

Permisos del Copilot coding agent

El Copilot coding agent opera bajo un modelo de **mínimo privilegio**:

Permiso	Descripción	Configuración
Creación de ramas	El agente puede crear ramas en el repo	Habilitado por defecto
Creación de PRs	El agente puede abrir Pull Requests	Habilitado por defecto
Merge directo	El agente puede hacer merge a main	✗ No permitido (por diseño)
Acceso a secretos	El agente puede leer secrets del repo	Limitado a los configurados explícitamente
Ejecución de comandos	El agente puede ejecutar comandos en terminal	Dentro de la sandbox únicamente
Acceso a red	El agente puede hacer peticiones de red	Controlado por firewall de la sandbox

Control de permisos en VS Code

En VS Code, los permisos de herramientas se gestionan de varias formas:

1. **Aprobación por sesión:** VS Code pide confirmación la primera vez que una herramienta se usa.
2. **Configuración en `settings.json`:**

```
{
  "github.copilot.chat.agent.tools": {
    "terminal": {
      "enabled": true,
      "requireConfirmation": true
    },
    "filesystem": {
      "enabled": true,
      "readOnly": false
    }
  }
}
```

3. **Listas de permitidos/bloqueados (allowlists):** Controlan qué servidores MCP están autorizados (ver sección 2.2).

Modelo de confianza por capas

Capa 1: Organización (GitHub Org)
- Políticas de seguridad globales
- MCP allowlists a nivel de org
- Restricciones de repos permitidos

Capa 2: Repositorio
- Branch protection rules
- CODEOWNERS
- copilot-instructions.md
- copilot-setup-steps.yml

Capa 3: Agente individual
- Herramientas permitidas
- Permisos de ejecución
- Restricciones de archivos/directorios

Aplicación práctica en GitHub

Escenario: Configurar un agente para un proyecto Node.js con base de datos PostgreSQL.

1. **Crear** `.github/copilot-instructions.md` con las convenciones del proyecto.
2. **Crear** `.github/copilot-setup-steps.yml` para instalar Node.js, dependencias y configurar la BD de test.
3. **Configurar permisos** asegurando que el agente no pueda modificar archivos de infraestructura.
4. **Agregar branch protection** para que los PRs del agente requieran aprobación humana y pasen CI.

Puntos clave para el examen

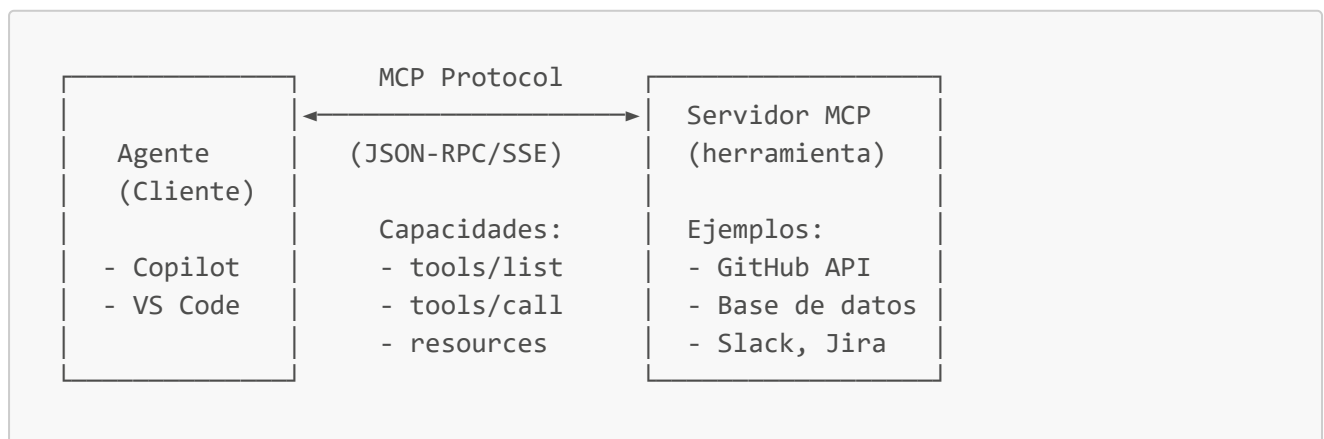
- ✂ `copilot-setup-steps.yml` configura el entorno ANTES de que el agente trabaje; no confundir con las instrucciones del agente.
- ✂ `.github/copilot-instructions.md` es para instrucciones de COMPORTAMIENTO; `copilot-setup-steps.yml` es para configuración de ENTORNO.
- ✂ El Copilot coding agent **nunca puede hacer merge directo** a la rama principal.
- ✂ Los permisos siguen el principio de **mínimo privilegio**.
- ✂ Las herramientas se seleccionan según un análisis de **capacidades requeridas vs. disponibles**.
- ✂ La configuración de herramientas opera en tres capas: organización → repositorio → agente individual.

2.2 Configuración de servidores MCP

Conceptos clave

¿Qué es MCP (Model Context Protocol)?

MCP (Model Context Protocol) es un protocolo abierto que estandariza cómo los modelos de IA se conectan con fuentes de datos y herramientas externas. Funciona como un **"USB-C para agentes de IA"**: un conector universal que permite al agente interactuar con cualquier servicio que implemente el protocolo.



Componentes fundamentales de MCP:

Componente	Descripción
Cliente MCP	El agente que consume las herramientas (Copilot, VS Code)
Servidor MCP	Proceso que expone herramientas y recursos via el protocolo MCP

Componente	Descripción
Transporte	Mecanismo de comunicación: stdio (local) o SSE/HTTP (remoto)
Herramientas (Tools)	Funciones que el servidor expone al agente para que las invoque
Recursos (Resources)	Datos contextuales que el servidor proporciona al agente
Prompts	Plantillas de instrucciones predefinidas que el servidor ofrece

Tipos de servidores MCP

Tipo	Transporte	Uso	Ejemplo
Local (stdio)	stdin/stdout	Desarrollo local, extensiones de VS Code	Servidor MCP de filesystem
Remoto (SSE/HTTP)	Server-Sent Events sobre HTTP	Servicios compartidos, APIs en la nube	GitHub Remote MCP Server

Agregar un servidor MCP como herramienta a un agente

Configuración en VS Code (**.vscode/mcp.json**)

El archivo **.vscode/mcp.json** es el estándar para configurar servidores MCP en VS Code a nivel de workspace:

```
// .vscode/mcp.json
{
  "servers": {
    "github": {
      "type": "sse",
      "url": "https://api.github.com/mcp",
      "headers": {
        "Authorization": "Bearer ${env:GITHUB_TOKEN}"
      }
    },
    "filesystem": {
      "type": "stdio",
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "${workspaceFolder}"
      ]
    },
    "postgres": {
      "type": "stdio",
      "command": "npx",
      "args": [
        "-y",
```

```

        "@modelcontextprotocol/server-postgres"
    ],
    "env": {
        "DATABASE_URL": "${env:DATABASE_URL}"
    }
}
}
}
}

```

Configuración en `settings.json` de VS Code (nivel usuario)

```

// settings.json (usuario o workspace)
{
  "mcp": {
    "servers": {
      "brave-search": {
        "type": "stdio",
        "command": "npx",
        "args": ["-y", "@anthropic/mcp-server-brave-search"],
        "env": {
          "BRAVE_API_KEY": "${env:BRAVE_API_KEY}"
        }
      }
    }
  }
}

```

Configuración para Copilot coding agent (nube)

Para el Copilot coding agent que se ejecuta en la nube, los servidores MCP se configuran en el archivo `.github/copilot-setup-steps.yml` y se referencian en la configuración del agente:

```

# .github/copilot-setup-steps.yml
steps:
  - name: Instalar servidor MCP de filesystem
    run: npm install -g @modelcontextprotocol/server-fileSystem

  - name: Instalar servidor MCP personalizado
    run: pip install my-custom-mcp-server

```

Configuración de un servidor MCP remoto de GitHub

GitHub proporciona un **servidor MCP remoto oficial** en <https://api.github.com/mcp> que expone las capacidades de la API de GitHub como herramientas MCP:

Herramientas disponibles en el MCP remoto de GitHub:

- Gestión de repositorios (crear, listar, buscar)
- Gestión de issues y PRs (crear, leer, actualizar, comentar)
- Gestión de archivos (leer, crear, actualizar contenido)
- Búsqueda de código
- Gestión de ramas y tags
- Acceso a GitHub Actions (workflows, runs)
- Gestión de releases

Configuración del servidor MCP remoto de GitHub:

```
// .vscode/mcp.json
{
  "servers": {
    "github": {
      "type": "sse",
      "url": "https://api.github.com/mcp",
      "headers": {
        "Authorization": "Bearer ${env:GITHUB_TOKEN}"
      }
    }
  }
}
```

Autenticación: El servidor MCP remoto de GitHub utiliza **tokens de acceso personal (PAT)** o **GitHub App tokens** para autenticación. El token necesita los scopes adecuados según las operaciones que el agente necesite realizar:

Scope del token	Operaciones permitidas
repo	Acceso completo a repositorios privados
read:org	Leer información de la organización
workflow	Gestionar GitHub Actions workflows
admin:org	Administrar la organización

Flujo de autenticación con OAuth (servidores remotos):

Para servidores MCP remotos que soportan OAuth, VS Code puede manejar el flujo de autenticación automáticamente:

```
{
  "servers": {
    "github-oauth": {
      "type": "sse",
      "url": "https://api.github.com/mcp",
      "auth": "oauth"
    }
  }
}
```

```
}  
}
```

Configuración de los registros de MCP

Los **registros de MCP (MCP registries)** son directorios centralizados que permiten descubrir y compartir servidores MCP disponibles. Funcionan como un "marketplace" de herramientas para agentes.

Concepto clave: Un registro MCP permite a las organizaciones:

1. **Descubrir** servidores MCP disponibles para diferentes casos de uso.
2. **Centralizar** la gestión de versiones y configuraciones.
3. **Compartir** servidores MCP entre equipos de la organización.
4. **Auditar** qué servidores MCP se están usando y por quién.

Configuración de un registro MCP:

```
// .vscode/mcp.json  
{  
  "registries": {  
    "my-org-registry": {  
      "url": "https://registry.myorg.com/mcp",  
      "auth": {  
        "type": "bearer",  
        "token": "${env:REGISTRY_TOKEN}"  
      }  
    }  
  },  
  "servers": {  
    "from-registry": {  
      "registry": "my-org-registry",  
      "name": "custom-tool-server",  
      "version": "1.2.0"  
    }  
  }  
}
```

Beneficios de usar registros:

- **Versiónado:** Cada servidor tiene versiones semánticas, permitiendo actualizaciones controladas.
- **Descubrimiento:** Los desarrolladores pueden explorar herramientas disponibles sin configuración manual.
- **Gobernanza:** La organización controla qué herramientas están aprobadas.

Configurar listas de permitidos de MCP (MCP Allowlists)

Las **listas de permitidos (allowlists)** son un mecanismo de seguridad a nivel de organización que controla qué servidores MCP pueden ser utilizados por los agentes. Son fundamentales para la

gobernanza de herramientas.

¿Por qué son necesarias?

- Prevenir que agentes se conecten a servidores MCP no autorizados.
- Garantizar que solo se usen herramientas aprobadas por seguridad.
- Cumplir con políticas de compliance y privacidad de datos.
- Evitar la exfiltración de datos a través de servidores MCP maliciosos.

Configuración a nivel de organización en GitHub:

Las allowlists se configuran en la configuración de la organización de GitHub:

1. Ir a **Organization Settings** → **Copilot** → **Políticas** → **MCP**.
2. Definir la lista de servidores MCP permitidos.
3. Configurar si se bloquean todos los servidores no listados (modo estricto) o si se permiten con advertencia (modo permisivo).

Niveles de control:

Nivel	Descripción	Configuración
Permitir todo	Cualquier servidor MCP puede ser usado	Sin restricciones (no recomendado para producción)
Lista de permitidos	Solo servidores en la lista pueden ser usados	Especificar URLs/nombres permitidos
Bloquear todo	Ningún servidor MCP externo puede ser usado	Solo herramientas integradas

Ejemplo de política de allowlist:

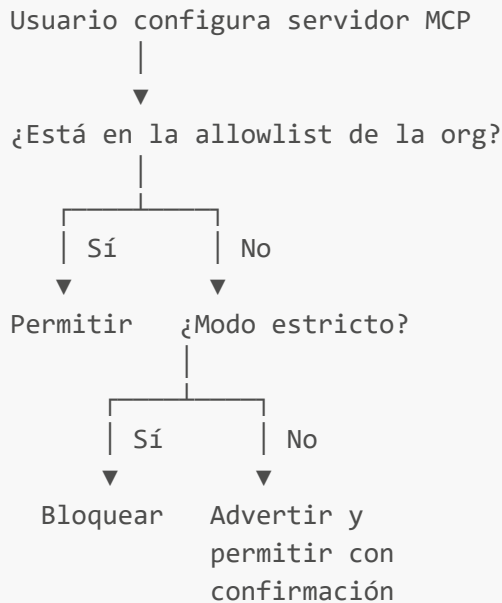
```
// Política de organización (configurada en GitHub)
{
  "mcp_policy": {
    "mode": "allowlist",
    "allowed_servers": [
      {
        "url": "https://api.github.com/mcp",
        "name": "GitHub Official MCP",
        "approved_by": "security-team",
        "approved_date": "2025-01-15"
      },
      {
        "url": "https://mcp.internal.myorg.com/*",
        "name": "Internal MCP Servers",
        "approved_by": "platform-team",
        "approved_date": "2025-02-01"
      }
    ]
  }
}
```

```

    "blocked_servers": [
      {
        "pattern": "*.untrusted-domain.com",
        "reason": "Dominio no verificado"
      }
    ]
  }
}

```

Flujo de validación de allowlists:



Aplicación práctica en GitHub

Escenario: Configurar MCP para un equipo de desarrollo que necesita acceso a GitHub API, una base de datos PostgreSQL y un sistema de tickets Jira.

1. **Configurar** `.vscode/mcp.json` con los tres servidores MCP.
2. **Solicitar que el equipo de seguridad apruebe** los servidores en la allowlist de la organización.
3. **Configurar variables de entorno** para tokens y credenciales (nunca hardcodear).
4. **Documentar en** `copilot-instructions.md` qué herramientas MCP están disponibles y para qué usarlas.
5. **Probar la conectividad** con cada servidor MCP antes de asignar tareas al agente.

Puntos clave para el examen

- **MCP** es un protocolo abierto que estandariza la conexión entre agentes y herramientas externas.
- Hay dos tipos de transporte: **stdio** (local) y **SSE/HTTP** (remoto).
- El servidor MCP remoto de GitHub está en <https://api.github.com/mcp>.
- `.vscode/mcp.json` es el archivo estándar para configurar servidores MCP en VS Code a nivel workspace.

- Los **registros MCP** permiten descubrir y compartir servidores MCP de forma centralizada.
- Las **allowlists** son controles de seguridad a nivel de organización que restringen qué servidores MCP pueden usarse.
- **Nunca hardcodear credenciales** en la configuración MCP; usar variables de entorno (`${env:VARIABLE}`).
- La autenticación del MCP remoto de GitHub usa **PAT** o **GitHub App tokens** con los scopes apropiados.
- Las allowlists pueden operar en modo **estricto** (bloquear todo no listado) o **permisivo** (advertir pero permitir).

2.3 Integración de agentes en entornos de desarrollo

Conceptos clave

Evaluación del contexto de ejecución de un agente

El **contexto de ejecución** define el entorno en el que el agente opera. Evaluarlo correctamente es fundamental para garantizar que el agente tenga acceso a las herramientas, datos y permisos que necesita.

Factores del contexto de ejecución:

Factor	Copilot coding agent (nube)	Copilot en VS Code (local)
Entorno	VM sandbox efímera en GitHub	Máquina local del desarrollador
Acceso al código	Clone del repositorio en la VM	Workspace abierto en VS Code
Red	Controlada por firewall de GitHub	Red local del desarrollador
Persistencia	Efímera (se destruye al terminar)	Persistente (workspace local)
Secretos	Secrets configurados en el repo/org	Variables de entorno locales
Herramientas	Definidas en <code>copilot-setup-steps.yml</code>	Extensiones y MCP configurados
Permisos	Token del agente con scopes limitados	Permisos del usuario local

Evaluación del contexto — preguntas clave:

1. **¿Dónde se ejecuta?** → Sandbox en la nube vs. máquina local.
2. **¿Qué puede ver?** → Repositorio completo vs. workspace parcial.
3. **¿Qué puede hacer?** → Permisos de escritura, ejecución, red.
4. **¿Cuánto tiempo persiste?** → Efímero vs. persistente.
5. **¿Qué dependencias necesita?** → Runtime, bibliotecas, servicios.

Sandbox del Copilot coding agent:

El Copilot coding agent se ejecuta en una **VM sandbox aislada** proporcionada por GitHub. Características:

- Sistema operativo Linux (Ubuntu-based)

- Acceso de red controlado por **firewall**
- Clone automático del repositorio
- Entorno efímero: se destruye al finalizar la tarea
- Los pasos de configuración se definen en `copilot-setup-steps.yml`

Configuración del firewall de la sandbox:

```
# Configuración de firewall para el Copilot coding agent
# Se configura en la organización de GitHub

# Dominios permitidos por defecto:
# - github.com
# - api.github.com
# - npm registry (registry.npmjs.org)
# - PyPI (pypi.org)

# Dominios adicionales (configurables por la org):
# - registros de paquetes internos
# - APIs internas necesarias
# - Servidores MCP remotos aprobados
```

Configuración del ámbito de un agente en un repositorio específico

El **ámbito (scope)** de un agente define los límites de su actuación dentro de un repositorio. Configurar correctamente el ámbito es esencial para evitar que el agente modifique código que no debería tocar.

Mecanismos de control de ámbito:

1. Instrucciones de repositorio (`.github/copilot-instructions.md`)

```
<!-- .github/copilot-instructions.md -->

## Ámbito de actuación

### Archivos que PUEDES modificar:
- `src/**/*.ts` – Código fuente de la aplicación
- `tests/**/*.test.ts` – Tests unitarios y de integración
- `docs/**/*.md` – Documentación

### Archivos que NO debes modificar:
- `.github/workflows/**` – Workflows de CI/CD
- `infrastructure/**` – Configuración de infraestructura
- `*.config.js` – Archivos de configuración raíz
- `.env*` – Variables de entorno
```

2. CODEOWNERS

El archivo **CODEOWNERS** permite requerir aprobaciones específicas según los archivos modificados:

```
# .github/CODEOWNERS

# Código de la aplicación – puede ser revisado por el equipo
/src/                                @team/developers

# Infraestructura – requiere aprobación del equipo de plataforma
/infrastructure/                    @team/platform
/.github/workflows/                @team/platform

# Seguridad – requiere aprobación del equipo de seguridad
/src/auth/                          @team/security
/src/crypto/                        @team/security
```

3. Branch protection rules

Configuración de branch protection para main:

- ☒ Require pull request reviews before merging
- ☒ Require status checks to pass before merging
 - Required checks: [build, test, lint, security-scan]
- ☒ Require conversation resolution before merging
- ☒ Restrict pushes that create matching refs
- ☒ Include administrators

4. Rulesets (reglas de repositorio)

Los rulesets de GitHub proporcionan un control más granular que las branch protection rules:

```
# Ejemplo conceptual de ruleset
ruleset:
  name: "protect-infrastructure"
  target: branch
  conditions:
    ref_name:
      include: ["refs/heads/main", "refs/heads/release/*"]
  rules:
    - type: pull_request
      parameters:
        required_approving_review_count: 2
        require_code_owner_review: true
    - type: required_status_checks
      parameters:
        strict_required_status_checks_policy: true
        required_status_checks:
          - context: "CI / build"
```

- context: "CI / test"
- context: "security / scan"

Configuración de un agente que se va a invocar en un flujo de trabajo de CI

Los agentes pueden ser invocados como parte de un pipeline de CI/CD usando GitHub Actions. Esto permite **automatizar tareas de desarrollo** como respuesta a eventos del repositorio.

Patrones de integración con CI:

Patrón 1: Validar la salida del agente con CI

```
# .github/workflows/validate-agent-pr.yml
name: Validar PR del agente

on:
  pull_request:
    types: [opened, synchronize]

jobs:
  validate:
    # Solo ejecutar en PRs creadas por el Copilot coding agent
    if: github.actor == 'copilot[bot]' || github.actor == 'github-actions[bot]'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Instalar dependencias
        run: npm ci

      - name: Ejecutar linter
        run: npm run lint

      - name: Ejecutar tests
        run: npm test

      - name: Verificar cobertura
        run: npm run test:coverage -- --min-coverage=80

      - name: Análisis de seguridad
        run: npm audit --audit-level=high

      - name: Comentar resultado en el PR
        if: failure()
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.createComment({
              issue_number: context.issue.number,
              owner: context.repo.owner,
```



```
    repo: context.repo.repo,  
    body: '❌ La validación de CI falló. Por favor revisa los logs.'  
  })
```

Patrón 2: Disparar un agente desde CI

```
# .github/workflows/auto-fix-lint.yml  
name: Auto-fix con agente  
  
on:  
  pull_request:  
    types: [opened]  
  
jobs:  
  auto-fix:  
    runs-on: ubuntu-latest  
    permissions:  
      contents: write  
      pull-requests: write  
    steps:  
      - uses: actions/checkout@v4  
        with:  
          ref: ${ github.head_ref }  
  
      - name: Ejecutar linter con auto-fix  
        run: |  
          npm ci  
          npm run lint:fix  
          npm run format  
  
      - name: Verificar cambios  
        id: changes  
        run: |  
          if [ -n "$(git diff --name-only)" ]; then  
            echo "has_changes=true" >> $GITHUB_OUTPUT  
          fi  
  
      - name: Commit fixes  
        if: steps.changes.outputs.has_changes == 'true'  
        run: |  
          git config user.name "github-actions[bot]"  
          git config user.email "github-actions[bot]@users.noreply.github.com"  
          git add .  
          git commit -m "fix: auto-fix linting issues"  
          git push
```

Patrón 3: Asignar Copilot a issues automáticamente

```
# .github/workflows/assign-copilot.yml
name: Asignar Copilot a issues etiquetados

on:
  issues:
    types: [labeled]

jobs:
  assign-copilot:
    if: github.event.label.name == 'copilot-task'
    runs-on: ubuntu-latest
    permissions:
      issues: write
    steps:
      - name: Asignar Copilot al issue
        uses: actions/github-script@v7
        with:
          script: |
            await github.rest.issues.addAssignees({
              owner: context.repo.owner,
              repo: context.repo.repo,
              issue_number: context.issue.number,
              assignees: ['copilot']
            });
```

Configurar un agente para que utilice un ámbito basado en ramas

El **ámbito basado en ramas** permite que el agente trabaje en ramas específicas con reglas y permisos diferenciados.

Modelo de ramas para agentes:

```
main (protegida)
├── develop (integración)
│   ├── feature/copilot-123 ← Rama creada por el agente
│   ├── feature/copilot-456 ← Rama creada por el agente
│   └── feature/manual-789 ← Rama creada por un humano
└── release/* (protegida)
```

Convención de nombres de ramas del agente:

El Copilot coding agent utiliza una convención de nombres predecible:

- `copilot/fix-{issue-number}-{slug}` — para correcciones de bugs
- `copilot/feature-{issue-number}-{slug}` — para nuevas características

Configuración de protección basada en ramas:

```
# Ejemplo de ruleset para ramas del agente
ruleset:
  name: "copilot-branches"
  target: branch
  conditions:
    ref_name:
      include: ["refs/heads/copilot/*"]
  rules:
    - type: pull_request
      parameters:
        required_approving_review_count: 1
        require_code_owner_review: true
    - type: required_status_checks
      parameters:
        required_status_checks:
          - context: "CI / build"
          - context: "CI / test"
```

Control de acceso por rama:

Rama	¿Quién puede pushear?	¿Requiere PR?	¿Requiere reviews?
main	Nadie (solo merge)	Sí	2 reviews mínimo
develop	Equipo de desarrollo	Sí	1 review mínimo
copilot/*	Solo el agente	Sí (hacia develop/main)	1 review mínimo
feature/*	Desarrolladores	Sí	1 review mínimo

Habilitar a un agente para que realice acciones autónomas

Las acciones autónomas son aquellas que el agente realiza sin intervención humana directa. Requieren un balance cuidadoso entre **autonomía y supervisión**.

Acciones autónomas permitidas por defecto:

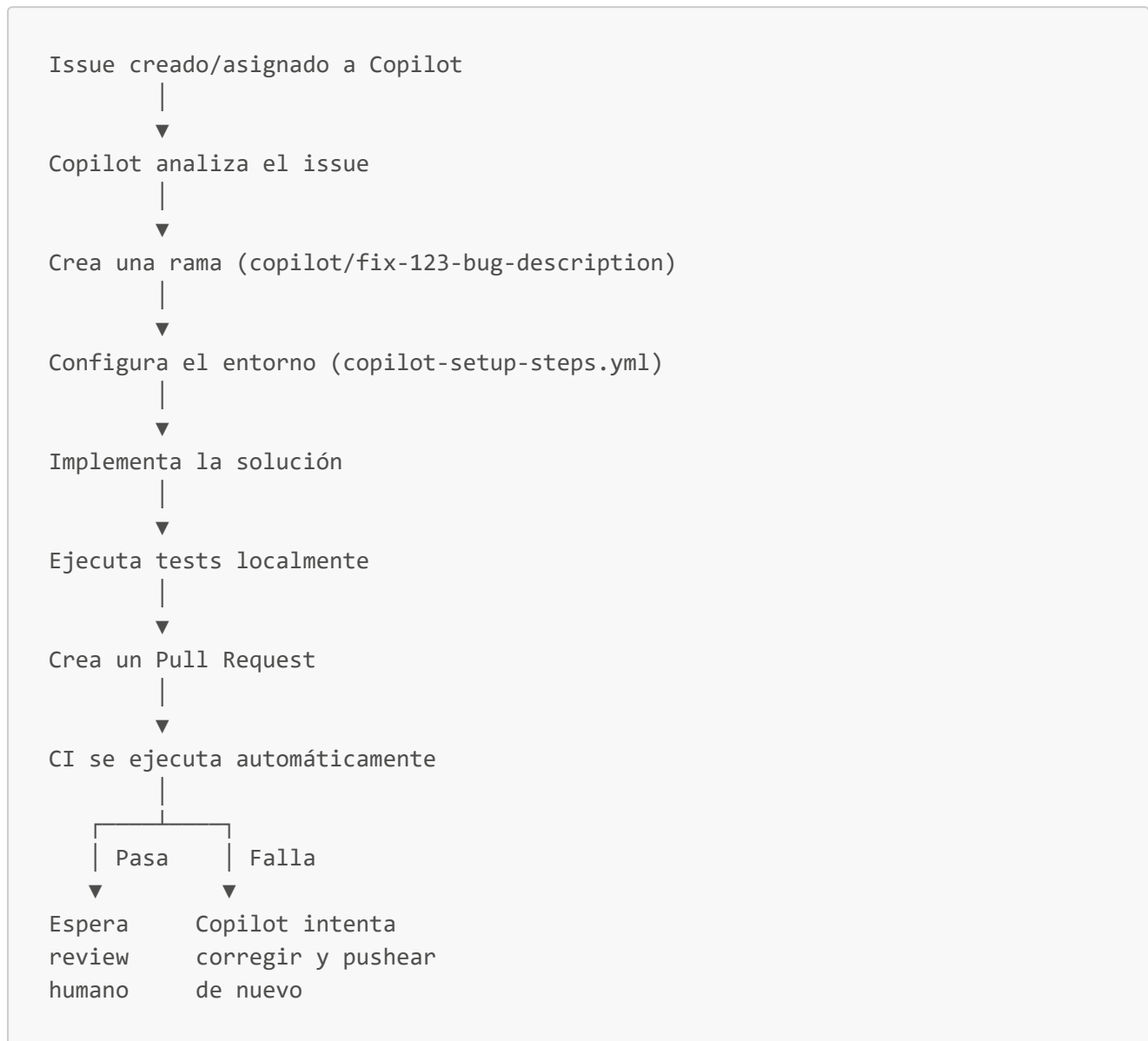
Acción	Nivel de autonomía	Supervisión requerida
Crear rama	☒ Totalmente autónoma	Ninguna
Escribir código	☒ Totalmente autónoma	Revisión posterior en el PR
Ejecutar tests	☒ Totalmente autónoma	Los resultados se reflejan en CI
Crear PR	☒ Totalmente autónoma	El PR requiere revisión humana
Responder a review comments	☒ Totalmente autónoma	El reviewer ve las respuestas
Hacer merge	☒ No permitido	Siempre requiere aprobación humana

Acción	Nivel de autonomía	Supervisión requerida
Modificar workflows de CI	✗ No recomendado	Requiere aprobación especial
Acceder a producción	✗ Prohibido	No aplica

Configuración para habilitar acciones autónomas del Copilot coding agent:

1. **En la organización:** Habilitar Copilot coding agent en la política de la organización.
2. **En el repositorio:** Asegurarse de que el repositorio permite al agente crear ramas y PRs.
3. **Asignar el agente:** Usar `@copilot` en un issue o asignar Copilot directamente.

Flujo de trabajo autónomo:



Configuración de un agente para controlar restricciones específicas del entorno

Las restricciones del entorno son limitaciones técnicas o de política que el agente debe respetar.

Restricciones comunes y su configuración:

1. Restricciones de red (firewall)

```
# Configuración de firewall del Copilot coding agent
# (se configura en Organization Settings → Copilot → Networking)

# Dominios permitidos para el agente en la sandbox:
allowed_domains:
  - "*.github.com"
  - "*.npmjs.org"
  - "*.pypi.org"
  - "api.internal.mycompany.com"
  - "registry.internal.mycompany.com"

# Dominios bloqueados:
# Todo lo que no esté en la lista de permitidos está bloqueado por defecto
```

2. Restricciones de tiempo

El Copilot coding agent tiene un **límite de tiempo** para completar su tarea. Si no termina, la sesión se cancela y el agente reporta lo que pudo hacer.

3. Restricciones de recursos

```
# copilot-setup-steps.yml – gestionar restricciones de recursos
steps:
  - name: Verificar recursos disponibles
    run: |
      echo "Memoria disponible:"
      free -h
      echo "Espacio en disco:"
      df -h
      echo "CPUs disponibles:"
      nproc

  - name: Configurar límites de Node.js
    run: export NODE_OPTIONS="--max-old-space-size=4096"
```

4. Restricciones de acceso a archivos

```
<!-- .github/copilot-instructions.md -->

## Restricciones de acceso

### Archivos de solo lectura (no modificar):
- `package-lock.json` – solo se actualiza vía `npm install`
```

- ``prisma/migrations/**`` — solo se generan con ``npx prisma migrate dev``
- ``.github/workflows/**`` — gestionados por el equipo de plataforma

Directorios restringidos:

- ``/secrets/`` — nunca acceder a este directorio
- ``/production-config/`` — configuración de producción, no tocar

5. Restricciones de dependencias

```
<!-- .github/copilot-instructions.md -->
```

Política de dependencias

- NO instalar paquetes nuevos sin justificación explícita
- NO usar dependencias con licencia GPL en este proyecto (MIT/Apache only)
- NO actualizar dependencias mayor sin aprobación
- Verificar vulnerabilidades con ``npm audit`` antes de agregar dependencias

Aplicación práctica en GitHub

Escenario completo: Configurar un repositorio para que el Copilot coding agent pueda trabajar de forma segura y autónoma.

1. **Crear `.github/copilot-instructions.md`** — definir el ámbito, restricciones y convenciones.
2. **Crear `.github/copilot-setup-steps.yml`** — configurar el entorno de ejecución.
3. **Configurar CODEOWNERS** — proteger archivos sensibles con revisores obligatorios.
4. **Configurar branch protection** — requerir CI y reviews en PRs del agente.
5. **Configurar firewall** — permitir solo los dominios necesarios.
6. **Configurar MCP allowlists** — restringir herramientas externas a las aprobadas.
7. **Crear un issue de prueba** — asignar a Copilot y verificar el flujo completo.

Puntos clave para el examen

- ✂ El Copilot coding agent se ejecuta en una **VM sandbox efímera** en la nube de GitHub.
- ✂ `copilot-setup-steps.yml` configura el entorno de la sandbox (dependencias, runtime, herramientas).
- ✂ El **firewall de la sandbox** controla el acceso de red; por defecto solo permite dominios de GitHub y registros de paquetes.
- ✂ El ámbito del agente se controla mediante: **instrucciones** (`copilot-instructions.md`), **CODEOWNERS**, **branch protection rules** y **rulesets**.
- ✂ El agente puede crear ramas y PRs autónomamente, pero **nunca puede hacer merge directo**.
- ✂ La integración con CI se logra con **GitHub Actions workflows** que validan la salida del agente.
- ✂ Las ramas del agente siguen la convención `copilot/{tipo}-{issue}-{descripción}`.
- ✂ Las restricciones de entorno incluyen: red (firewall), tiempo, recursos, acceso a archivos y dependencias.
- ✂ Los **rulesets** proporcionan control más granular que las branch protection rules.

- ✂ Un agente en CI puede ser invocado en respuesta a eventos como `pull_request`, `issues`, `push`, etc.

2.4 Operar agentes con rutas de ejecución seguras y control robusto de errores

Conceptos clave

¿Por qué es crítico el control de errores en agentes?

A diferencia del software tradicional donde los errores son generalmente deterministas, los agentes de IA pueden fallar de formas **impredecibles y variadas**:

- Generar código que no compila
- Hacer suposiciones incorrectas sobre la estructura del proyecto
- Entrar en bucles de reintentos infinitos
- Modificar archivos incorrectos
- Ignorar errores silenciosamente

Por eso, se necesita un enfoque de **defensa en profundidad** con múltiples capas de protección:

Capa 1: Prevención
- Instrucciones claras
- Restricciones de ámbito
- Validación de entrada

Capa 2: Detección
- CI/CD (tests, linters)
- Checks automáticos
- Monitoreo de acciones

Capa 3: Recuperación
- Reintentos con backoff
- Rollbacks automáticos
- Revert de commits

Capa 4: Escalación
- Notificación a humanos
- Comentarios en PRs
- Asignación a reviewers
- Bloqueo de merge

Implementación del control de errores

Errores en el entorno de ejecución

```
# copilot-setup-steps.yml – con control de errores
steps:
  - name: Instalar dependencias
    run: |
      npm ci || {
        echo "Error: Fallo la instalación de dependencias"
        echo "Intentando con npm install..."
        npm install
      }

  - name: Verificar entorno
    run: |
      # Verificar que las herramientas necesarias están disponibles
      command -v node || { echo "Node.js no encontrado"; exit 1; }
      command -v npm || { echo "npm no encontrado"; exit 1; }
      node --version
      npm --version
```

Errores en la ejecución del agente

El Copilot coding agent tiene mecanismos internos de manejo de errores:

1. **Auto-corrección:** Si un test falla, el agente intenta corregir el código y ejecutar de nuevo.
2. **Reporte de errores:** Si no puede resolver el error, lo documenta en el PR.
3. **Abandono controlado:** Si la tarea no se puede completar, el agente crea el PR con lo que tiene e indica qué falta.

Patrones de manejo de errores en workflows de CI

```
# .github/workflows/agent-error-handling.yml
name: Manejo de errores del agente

on:
  pull_request:
    types: [opened, synchronize]

jobs:
  validate:
    if: github.actor == 'copilot[bot]'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Instalar dependencias
        run: npm ci

      - name: Ejecutar tests
        id: tests
```



```

run: npm test
continue-on-error: true

- name: Ejecutar linter
  id: lint
  run: npm run lint
  continue-on-error: true

- name: Reportar resultados
  if: steps.tests.outcome == 'failure' || steps.lint.outcome == 'failure'
  uses: actions/github-script@v7
  with:
    script: |
      let body = '## ⚠ Resultados de validación\n\n';

      if ('${{ steps.tests.outcome }}' === 'failure') {
        body += '❌ **Tests fallaron** – revisa los logs de CI\n';
      } else {
        body += '✅ **Tests pasaron**\n';
      }

      if ('${{ steps.lint.outcome }}' === 'failure') {
        body += '❌ **Linting falló** – revisa los logs de CI\n';
      } else {
        body += '✅ **Linting pasó**\n';
      }

      body += '\n@copilot por favor corrige los errores indicados.';

      github.rest.issues.createComment({
        issue_number: context.issue.number,
        owner: context.repo.owner,
        repo: context.repo.repo,
        body: body
      });

- name: Fallar el job si hay errores
  if: steps.tests.outcome == 'failure' || steps.lint.outcome == 'failure'
  run: exit 1

```

Implementar reintentos

Los **reintentos (retries)** son un patrón fundamental para manejar errores transitorios. En el contexto de agentes, los reintentos se aplican a nivel de:

1. **Operaciones del agente** (e.g., llamadas a API que fallan por timeout).
2. **CI/CD** (e.g., re-ejecutar tests que son flaky).
3. **Tarea completa** (e.g., pedir al agente que reintente la solución).

Patrón de reintento con backoff exponencial

Intento 1 → Fallo → Esperar 1s
Intento 2 → Fallo → Esperar 2s
Intento 3 → Fallo → Esperar 4s
Intento 4 → Fallo → Esperar 8s
Intento 5 → Fallo → ESCALAR A HUMANO

Reintentos en GitHub Actions

```
# Reintento de tests flaky
- name: Ejecutar tests con reintentos
  uses: nick-fields/retry@v3
  with:
    timeout_minutes: 10
    max_attempts: 3
    retry_wait_seconds: 30
    command: npm test

# Reintento manual del workflow completo
# Se puede re-ejecutar desde la UI de GitHub Actions
# o programáticamente:
- name: Solicitar re-ejecución si falla
  if: failure()
  uses: actions/github-script@v7
  with:
    script: |
      // Contar intentos previos
      const runs = await github.rest.actions.listWorkflowRuns({
        owner: context.repo.owner,
        repo: context.repo.repo,
        workflow_id: 'validate-agent-pr.yml',
        head_sha: context.sha,
      });

      const attempts = runs.data.workflow_runs.length;

      if (attempts < 3) {
        // Re-ejecutar el workflow
        await github.rest.actions.reRunWorkflow({
          owner: context.repo.owner,
          repo: context.repo.repo,
          run_id: context.runId,
        });
      } else {
        // Escalar a humano después de 3 intentos
        await github.rest.issues.createComment({
          issue_number: context.issue.number,
          owner: context.repo.owner,
          repo: context.repo.repo,
          body: '🚨 Los tests fallaron después de 3 intentos. Se requiere
```

```
intervención humana.'
    });
}
```

Reintentos del Copilot coding agent

El Copilot coding agent implementa reintentos internos:

1. **Auto-corrección de código:** Si la ejecución de tests falla, el agente analiza los errores y modifica el código.
2. **Iteraciones de review:** Si un reviewer solicita cambios, el agente los implementa y pushea de nuevo.
3. **Límite de iteraciones:** Existe un límite máximo de iteraciones para evitar bucles infinitos.

Implementar reversiones (rollbacks)

Los **rollbacks** son el mecanismo para deshacer cambios cuando algo sale mal. En el contexto de agentes, esto es especialmente importante porque el agente puede hacer múltiples commits antes de que alguien lo revise.

Rollback vía `git revert`

```
# Revertir el último commit del agente
git revert HEAD --no-edit

# Revertir un commit específico
git revert abc123 --no-edit

# Revertir un rango de commits
git revert HEAD~3..HEAD --no-edit
```

Rollback automatizado en CI

```
# .github/workflows/auto-rollback.yml
name: Auto-rollback si CI falla

on:
  push:
    branches: [develop]

jobs:
  validate-and-rollback:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
```

```

    fetch-depth: 0

- name: Ejecutar tests
  id: tests
  run: npm ci && npm test
  continue-on-error: true

- name: Auto-rollback si falla
  if: steps.tests.outcome == 'failure'
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "github-actions[bot]@users.noreply.github.com"

    # Identificar el último commit del agente
    LAST_COMMIT=$(git log -1 --format="%H")
    AUTHOR=$(git log -1 --format="%an")

    if [[ "$AUTHOR" == *"copilot"* ]] || [[ "$AUTHOR" == *"bot"* ]]; then
      echo "Revirtiendo commit del agente: $LAST_COMMIT"
      git revert $LAST_COMMIT --no-edit
      git push
    fi

- name: Notificar rollback
  if: steps.tests.outcome == 'failure'
  uses: actions/github-script@v7
  with:
    script: |
      github.rest.issues.createComment({
        issue_number: context.issue.number,
        owner: context.repo.owner,
        repo: context.repo.repo,
        body: '🔄 Se realizó un rollback automático del último commit del agente debido a fallos en los tests.'
      });

```

Rollback a nivel de PR

La forma más segura de manejar rollbacks en el contexto del agente es a nivel de **Pull Request**:

Estrategia	Cuándo usarla	Cómo
Cerrar el PR	El PR del agente es completamente incorrecto	Cerrar sin merge
Revert PR	El PR ya fue mergeado y causa problemas	Crear un PR de revert desde GitHub
Request changes	El PR tiene partes correctas pero necesita ajustes	Solicitar cambios al agente via review

Estrategia	Cuándo usarla	Cómo
Reset de rama	La rama del agente está en mal estado	<code>git reset --hard</code> al estado anterior

Implementación de rutas de escalación

Las **rutas de escalación** definen cómo se escala un problema desde el agente hasta un humano cuando el agente no puede resolverlo por sí mismo.

Matriz de escalación:

Nivel	Condición	Acción	Ejemplo
L0 — Auto-resolución	Error conocido con solución automatizable	Reintento automático	Test flaky → re-ejecutar
L1 — Notificación	Error que requiere atención pero no es urgente	Comentario en el PR	Dependencia deprecada
L2 — Solicitud de review	Error que el agente no puede resolver solo	Request review a un humano	Error de lógica de negocio
L3 — Bloqueo	Error crítico o de seguridad	Bloquear merge + notificación urgente	Vulnerabilidad detectada

Implementación de escalación en CI:

```
# .github/workflows/escalation.yml
name: Escalación de errores del agente

on:
  pull_request:
    types: [opened, synchronize]

jobs:
  check-and-escalate:
    if: github.actor == 'copilot[bot]'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Ejecutar validaciones
        id: validate
        run: |
          npm ci
          npm test 2>&1 | tee test-output.txt
          npm run lint 2>&1 | tee lint-output.txt
          npm audit --json 2>&1 | tee audit-output.txt
    continue-on-error: true
```

```

- name: Analizar severidad y escalar
  if: steps.validate.outcome == 'failure'
  uses: actions/github-script@v7
  with:
    script: |
      const fs = require('fs');

      // Analizar resultados de auditoría
      let auditData;
      try {
        auditData = JSON.parse(fs.readFileSync('audit-output.txt',
'utf8'));
      } catch (e) {
        auditData = { metadata: { vulnerabilities: {} } };
      }

      const vulns = auditData.metadata?.vulnerabilities || {};
      const hasCritical = (vulns.critical || 0) > 0;
      const hasHigh = (vulns.high || 0) > 0;

      if (hasCritical) {
        // L3: Bloqueo y notificación urgente
        await github.rest.issues.createComment({
          issue_number: context.issue.number,
          owner: context.repo.owner,
          repo: context.repo.repo,
          body: '🚨 **CRÍTICO**: Se detectaron vulnerabilidades
críticas.\n\n' +
              'Este PR ha sido bloqueado. Se requiere revisión
inmediata del equipo de seguridad.\n\n' +
              'cc @team/security'
        });

        // Solicitar review del equipo de seguridad
        await github.rest.pulls.requestReviewers({
          owner: context.repo.owner,
          repo: context.repo.repo,
          pull_number: context.issue.number,
          team_reviewers: ['security']
        });
      } else if (hasHigh) {
        // L2: Solicitar review
        await github.rest.issues.createComment({
          issue_number: context.issue.number,
          owner: context.repo.owner,
          repo: context.repo.repo,
          body: '⚠️ Se detectaron vulnerabilidades de severidad
alta.\n\n' +
              'Se solicita revisión antes de merge.\n\ncc
@team/developers'
        });
      }

```

```

    } else {
      // L1: Notificación
      await github.rest.issues.createComment({
        issue_number: context.issue.number,
        owner: context.repo.owner,
        repo: context.repo.repo,
        body: '⚠️ CI falló. Revisa los logs para más detalles.\n\n' +
              '@copilot por favor intenta corregir los errores.'
      });
    }
  }
}

```

Escalación del Copilot coding agent

El propio Copilot coding agent tiene mecanismos de escalación integrados:

1. **Comentarios en el PR:** El agente documenta qué intentó y por qué no pudo resolver el problema.
2. **Asignación de revisores:** El PR automáticamente solicita revisión según CODEOWNERS.
3. **Etiquetas automáticas:** Se pueden configurar workflows para agregar etiquetas como `needs-human-review`.
4. **Sesiones de seguimiento:** Un humano puede dejar un comentario pidiendo al agente que reintente con instrucciones adicionales.

Implementar la rastreabilidad y la responsabilidad de las acciones del agente

La **rastreabilidad (traceability)** es la capacidad de reconstruir **qué hizo el agente, cuándo, por qué y con qué resultado**. La **responsabilidad (accountability)** establece quién es responsable de las acciones del agente.

Mecanismos de rastreabilidad en GitHub

Mecanismo	Qué registra	Dónde verlo
Historial de git	Cada commit del agente con autor y timestamp	<code>git log --author="copilot[bot]"</code>
PR timeline	Todas las acciones del agente en el PR	Pestaña de conversación del PR
CI logs	Ejecuciones de tests, builds, checks	GitHub Actions → logs del workflow
Audit log (org)	Acciones administrativas del agente	Organization Settings → Audit log
Sesiones del agente	Razonamiento interno del agente	Enlace de sesión en el PR del agente

Rastreabilidad en commits

Los commits del Copilot coding agent contienen metadatos que permiten su identificación:

```

commit 1234567890abcdef1234567890abcdef12345678
Author: Copilot <copilot@github.com>
Date:   Thu Aug 7 12:34:56 2025 +0000

```

```
commit abc123def456
Author: copilot[bot] <copilot[bot]@users.noreply.github.com>
Date: Mon Jul 14 10:30:00 2025 +0000

    fix: resolve null pointer exception in user service

    - Added null check in UserService.getUser()
    - Updated tests to cover null case
    - Closes #123

Co-authored-by: Copilot <copilot[bot]@users.noreply.github.com>
```

Identificar commits del agente:

```
# Listar todos los commits del agente
git log --author="copilot\[bot\]" --oneline

# Contar commits del agente vs. humanos
git shortlog -sn --all | grep -i copilot

# Ver estadísticas de archivos modificados por el agente
git log --author="copilot\[bot\]" --stat
```

Audit logs de la organización

Los **audit logs** registran acciones a nivel de organización y permiten auditar las actividades del agente:

- Creación de ramas por el agente
- Pull Requests abiertos por el agente
- Cambios en configuración del agente
- Uso de herramientas MCP
- Acceso a recursos protegidos

Responsabilidad: ¿Quién es responsable?

Acción del agente	¿Quién es responsable?
Código generado	El reviewer que aprobó el PR
Issue asignado	La persona que asignó el issue al agente
Configuración del agente	El equipo que mantiene <code>copilot-instructions.md</code>
Políticas de seguridad	El equipo de seguridad de la organización
Merge del PR del agente	La persona que hizo click en merge

Principio fundamental: El agente es una herramienta, no un actor responsable. La responsabilidad recae en los humanos que:

1. **Configuran** al agente (instrucciones, permisos).
2. **Asignan** tareas al agente.
3. **Revisan y aprueban** la salida del agente.
4. **Hacen merge** del trabajo del agente.

Implementación de trazabilidad con GitHub Actions

```
# .github/workflows/agent-audit.yml
name: Auditoría de acciones del agente

on:
  pull_request:
    types: [opened, closed, merged]

jobs:
  audit:
    if: github.actor == 'copilot[bot]'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Generar reporte de auditoría
        uses: actions/github-script@v7
        with:
          script: |
            const { execSync } = require('child_process');

            // Obtener estadísticas del agente
            const commits = execSync(
              'git log --author="copilot" --oneline | wc -l'
            ).toString().trim();

            const filesChanged = execSync(
              'git diff --name-only origin/main...HEAD | wc -l'
            ).toString().trim();

            const insertions = execSync(
              'git diff --stat origin/main...HEAD | tail -1'
            ).toString().trim();

            const body = `## 📊 Reporte de auditoría del agente

            | Métrica | Valor |
            |---|---|
            | Commits realizados | ${commits} |
            | Archivos modificados | ${filesChanged} |
```

```

| Estadísticas | ${insertions} |
| Actor | ${context.actor} |
| Evento | ${context.eventName} |
| Timestamp | ${new Date().toISOString()} |
`;

github.rest.issues.createComment({
  issue_number: context.issue.number,
  owner: context.repo.owner,
  repo: context.repo.repo,
  body: body
});

```

Aplicación práctica en GitHub

Escenario completo de defensa en profundidad:

1. **Prevención:** Instrucciones claras en `copilot-instructions.md` + restricciones de ámbito.
2. **Detección:** CI con tests, linting, security scan en cada push del agente.
3. **Reintento:** El agente auto-corrige si CI falla; workflows con retry automático.
4. **Rollback:** Si los reintentos fallan, rollback automático del commit del agente.
5. **Escalación:** Si el rollback no resuelve, notificación al equipo humano con contexto.
6. **Trazabilidad:** Audit logs + reportes automáticos + historial de git para reconstruir cada acción.

Puntos clave para el examen

- ✂ El control de errores en agentes requiere un enfoque de **defensa en profundidad** (prevención → detección → recuperación → escalación).
- ✂ Los **reintentos** deben usar **backoff exponencial** para evitar sobrecargar servicios.
- ✂ El Copilot coding agent tiene **auto-corrección**: si los tests fallan, intenta corregir y re-ejecutar.
- ✂ El **rollback** más seguro es a nivel de PR: cerrar el PR sin merge o crear un PR de revert.
- ✂ `git revert` es la forma estándar de deshacer commits sin reescribir el historial.
- ✂ Las rutas de escalación van de **L0 (auto-resolución)** a **L3 (bloqueo + notificación urgente)**.
- ✂ La **trazabilidad** se logra con: historial de git, PR timeline, CI logs, audit logs y sesiones del agente.
- ✂ La **responsabilidad** recae en los humanos que configuran, asignan, revisan y aprueban el trabajo del agente.
- ✂ Los commits del agente se identifican por el autor `copilot[bot]`.
- ✂ El agente documenta su razonamiento en la **sesión enlazada** desde el PR.
- ✂ `continue-on-error: true` en GitHub Actions permite capturar y manejar fallos sin detener el workflow completo.

Resumen ejecutivo de la Sección 2

Subsección	Peso estimado	Temas críticos
------------	---------------	----------------

Subsección	Peso estimado	Temas críticos
2.1 Herramientas del agente	5-6%	<code>copilot-instructions.md</code> , <code>copilot-setup-steps.yml</code> , permisos, principio de mínimo privilegio
2.2 Servidores MCP	5-7%	Protocolo MCP, <code>stdio</code> vs <code>SSE</code> , servidor remoto de GitHub, registros, allowlists
2.3 Integración en entornos	5-7%	Sandbox del agente, firewall, ámbito por repo/rama, CI integration, acciones autónomas
2.4 Errores y seguridad	5-6%	Reintentos con backoff, rollbacks con <code>git revert</code> , escalación L0-L3, trazabilidad, responsabilidad

Recursos

- [Documentación oficial de GitHub Copilot](#)
- [GitHub Copilot coding agent](#)
- [Model Context Protocol \(MCP\)](#)
- [GitHub MCP Server](#)
- [Configuración de copilot-setup-steps.yml](#)
- [Customizing Copilot instructions](#)
- [GitHub Actions - Retry](#)
- [GitHub Branch Protection Rules](#)
- [GitHub Rulesets](#)
- [GitHub Audit Log](#)